

PhyloBot

A Pipeline for Comparative Analysis of Multiple Sequence Alignment Tools CLUSTALW and Tcoffee

Introduction



Programming side



MEGA PHYLOGENETIC TREES



Logic behind the methods



So, that didn't work...



Introduction

Why did we design PhyloBot?

We designed Phylo Bot to address limitations in traditional phylogenetic workflows, specifically the reliance on outdated tools like CLUSTALW, which has long been the standard for multiple sequence alignment (MSA) despite its known shortcomings. ClustalW struggles with accuracy – especially of diverse datasets, its sequential algorithm that requires a full tree, which produces higher-quality alignments by combining results from multiple methods. Our pipeline automates the process, ensuring reproducibility and efficiency, and highlights the differences between these tools. By modernizing outdated workflows, Phylo Bot provides more accurate and reliable phylogenetic analyses.



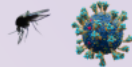
Modernizing CuPV-1 Phylogenetic Analysis with PhyloBot

For our project, we reanalyzed CuPV-1 alignments using PhyloBot to automate tasks, ensure reproducibility, and improve accuracy. By replacing ClustalW with T-Coffee for alignments and integrating tools like Nextflow, Nextflow, Nextflow, and Nextflow, we streamlined the pipeline for more reliable and efficient phylogenetic analysis.



In order to build and test our tool, we wanted to find a paper that matched our target audience. There were a couple criteria:

- The paper used ClustalW
- Aligned sequences had less than 80% average sequence identity
- Data used in the study was published and freely accessible.



Credit: Stock

and we found one....

Genetic characterization of novel picorna-like virus in *Culex* spp. mosquitoes from Mozambique (Costa et al., 2018)

This paper aims to identify and genetically characterize a novel picorna-like virus in *Culex* mosquitoes from Mozambique, exploring its evolutionary relationships and genetic features.

Sequences used had a predicted 38% identity.

ClustalW was used for multiple sequence alignment to identify conserved regions, infer phylogenetic relationships, and classify the virus within the picornavirus family.

Goals of the Project

This project aims to develop a bioinformatics pipeline tool that will demonstrate the difference between the quality of CLUSTALW and T-Coffee Alignments with easy data integration. It allows researchers to see the difference and may help them choose to replace their old methods with better ones.



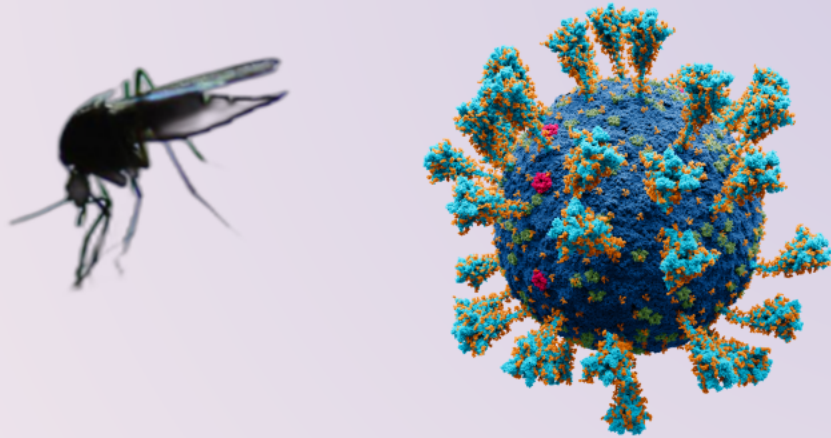
Why did we design PhyloBot?

We designed Phylo Bot to address limitations in traditional phylogenetic workflows, specifically the reliance on outdated tools like ClustalW, which has long been the standard for multiple sequence alignment (MSA) despite its known shortcomings. ClustalW struggles with accuracy in complex or diverse datasets, so we created a pipeline that replaces it with T-Coffee, which produces higher-quality alignments by combining results from multiple methods. Our pipeline automates the process, ensuring reproducibility and efficiency, and highlights the differences between these tools. By modernizing outdated workflows, Phylo Bot provides more accurate and reliable phylogenetic analyses.



In order to build and test our tool, we wanted to find a paper that matched our target audience. There were a couple criteria:

- The paper used CLUSTALW.
- Aligned sequences had less than 40% average sequence identity.
- Data used in the study was published and freely accessible.



and we found one....

Genetic characterization of novel
picorna-like virus in *Culex* spp.
mosquitoes from Mozambique

Cholleti et al., 2018

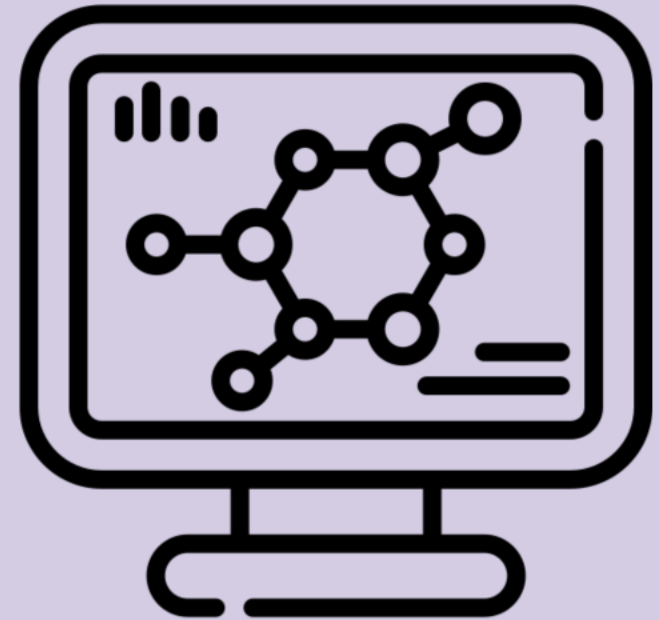
This paper aims to identify and genetically characterize a novel picorna-like virus in *Culex* mosquitoes from Mozambique, exploring its evolutionary relationships and genetic features.

Sequences used had a predicted 38% similarity

ClustalW was used for multiple sequence alignment to identify conserved regions, infer phylogenetic relationships, and classify the virus within the picornavirus family.

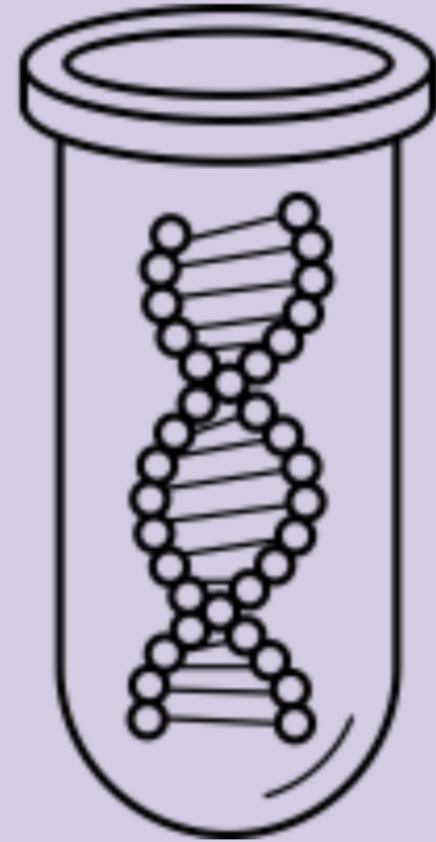
Modernizing CuPV-1 Phylogenetic Analysis with PhyloBot

For our project, we reanalyzed CuPV-1 alignments using PhyloBot to automate tasks, ensure reproducibility, and improve accuracy. By replacing ClustalW with T-Coffee for alignments and integrating tools like Nextflow, NCBI ORF Finder, MEGA, and Biopython, we streamlined the pipeline for more reliable and efficient phylogenetic analysis.



Goals of the Project

This project aims to develop a bioinformatics pipeline tool that will demonstrate the difference between the quality of CLUSTALW and T-Coffee Alignments with easy data integration. It allows researchers to see the difference and may help them choose to replace their old methods with better ones.



Logic behind the methods

Input Specifications: Accession Numbers

The tool requires sequence data input represented by accession numbers, which are unique identifiers assigned to biological sequences in databases. These numbers allow the tool to retrieve relevant sequences efficiently for analysis.

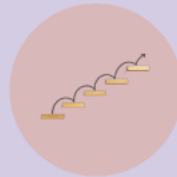


CLUSTALW: Progressive Approach

Pairwise Alignment: Clustal begins by calculating pairwise alignments between all sequences to generate a distance matrix.

Guide Tree Construction: A phylogenetic-like guide tree is created from the distance matrix, representing the similarities between sequences.

Progressive Alignment: The sequences are aligned in the order dictated by the guide tree. The two closest related sequences are aligned first, and then each sequence is added as according to its position in the guide tree.



T-Coffee: Consistency based approach

Pairwise Alignment with External Methods: T-Coffee generates pairwise alignments using multiple external methods (e.g., Clustal, LAISON), combining results into a consistency matrix.

Consistency-Based Scoring: Alignments are scored based on consistency with other pairwise alignments, ensuring that the alignment is globally coherent across all sequences.

Progressive Alignment: Sequences are aligned progressively following a guide tree, but the consistency matrix is used to improve accuracy at each step by resolving conflicts.



Output: Phylogenetic Trees!

The tool's output should consist of phylogenetic trees created by CLUSTALW and T-Coffee. This will allow the user to compare them side by side, assessing the bootstrap scores and phylogenetic placements.

Following the paper, this would be a manual step (not automated).



Input Specifications: Accession Numbers

The tool requires sequence data input represented by accession numbers, which are unique identifiers assigned to biological sequences in databases. These numbers allow the tool to retrieve relevant sequences efficiently for analysis.



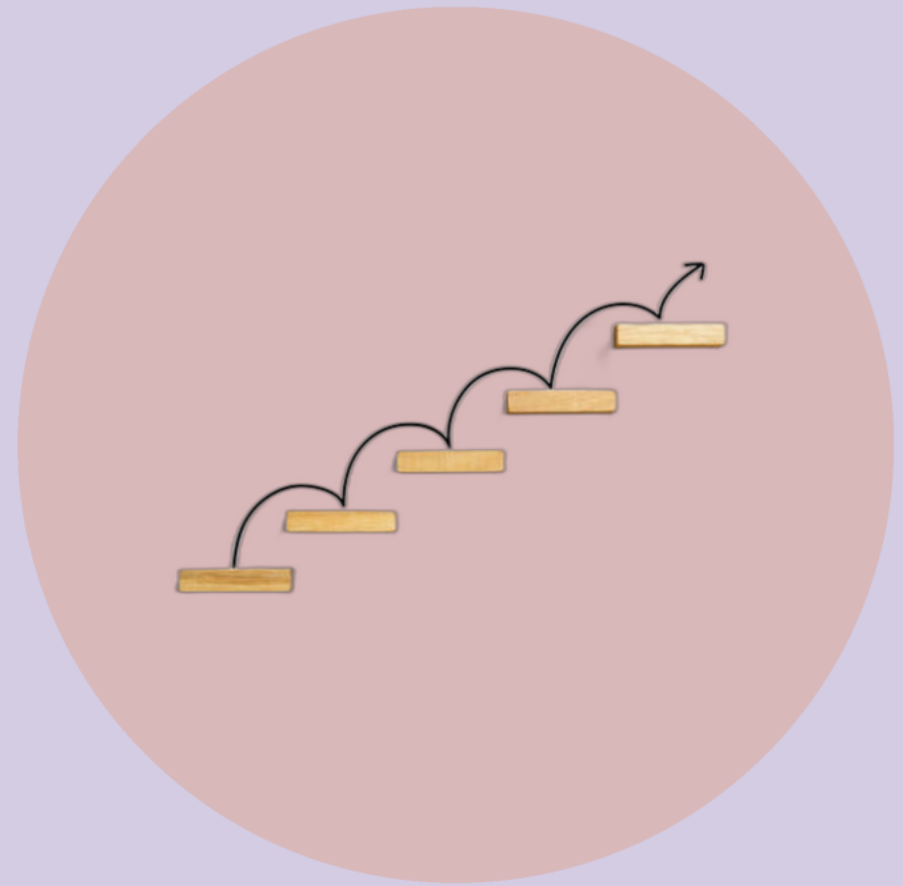
CLUSTALW:

Progressive Approach

Pairwise Alignment: Clustal begins by calculating pairwise alignments between all sequences to generate a distance matrix.

Guide Tree Construction: A phylogenetic-like guide tree is created from the distance matrix, representing the similarities between sequences.

Progressive Alignment: The sequences are aligned in the order dictated by the guide tree. The two closest related sequences are aligned first, and then each sequence is added according to its position in the guide tree.



T-Coffee:

Consistency based approach

Pairwise Alignment with External Methods: T-Coffee generates pairwise alignments using multiple external methods (e.g., Clustal, LALIGN), combining results into a consistency matrix.

Consistency-Based Scoring: Alignments are scored based on consistency with other pairwise alignments, ensuring that the alignment is globally coherent across all sequences.

Progressive Alignment: Sequences are aligned progressively following a guide tree, but the consistency matrix is used to improve accuracy at each step by resolving conflicts.

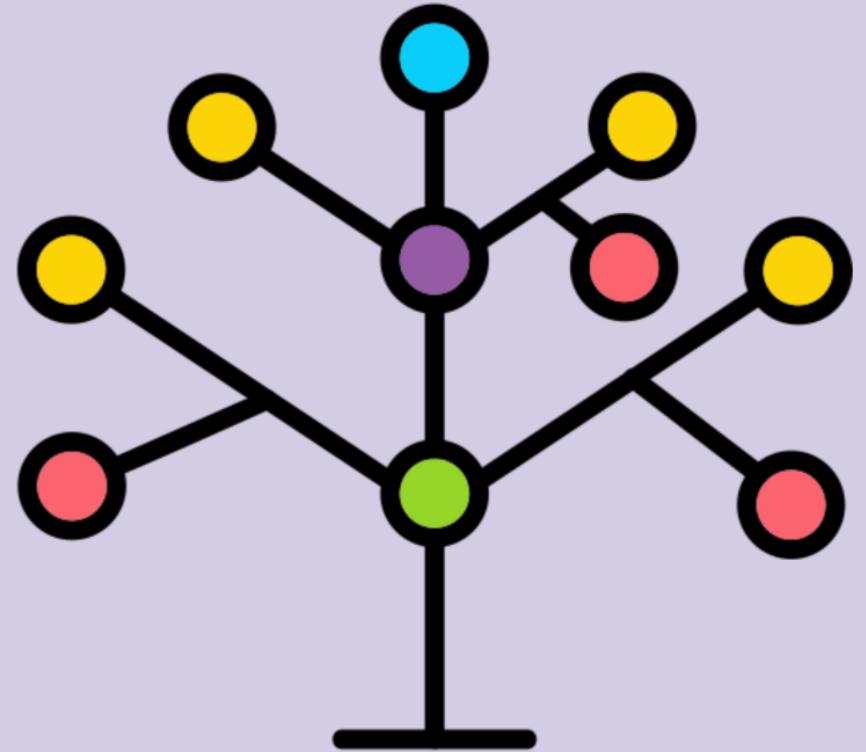


Output:

Phylogenetic Trees!

The tool's output should consist of phylogenetic trees created by CLUSTALW and T-Coffee. This will allow the user to compare them side by side, assessing the bootstrap scores and phylogenetic placements.

Following the paper, this would be a manual step (not automated).



Programming side



GETSEQS.sh

This Bash script automates the retrieval of protein sequences in FASTA format from NCBI using accession numbers. It:

1. Defines a list of accession numbers.
2. Uses a 'for' loop to fetch each sequence via the NCBI E-utilities API ('efetch.fcgi') with 'curl'.
3. Appends results to a file named 'sequences.fasta'.

Output: A FASTA file containing all the requested protein sequences.

```
#!/bin/bash
# GETSEQS.sh: Retrieve protein sequences from NCBI
# Usage: GETSEQS.sh <accession_numbers>

# Define accession numbers (you can add more here)
accessions="P00001 P00002 P00003"

# Create output file
output="sequences.fasta"

# Fetch sequences
for acc in $accessions; do
    curl -s "https://eutils.ncbi.nlm.nih.gov/efetch.fcgi?db=protein&rettype=fasta&retmode=text&accession=$acc" >> $output
done
```

CLUSTAL.sh

This Bash script uses ClustalW to perform a multiple sequence alignment of protein or nucleotide sequences. It takes 'trimmed_sequences.fasta' as the input file and generates an aligned output file named 'aligned_sequences.fasta' in FASTA format.

```
#!/bin/bash
# CLUSTAL.sh: Perform multiple sequence alignment using ClustalW
# Usage: CLUSTAL.sh <trimmed_sequences.fasta>

# Input file
input="trimmed_sequences.fasta"

# Run ClustalW
/usr/bin/clustalw -i $input -o aligned_sequences.fasta -m
```

CLEANRDP.py

This Python script removes gaps (-) from the trimmed RDP sequences to ensure clean, gap-free sequences. It:

1. Reads the sequences from 'trimmed_rdp.fasta' using Biopython's SeqIO.
2. Replaces gaps in each sequence with an empty string.
3. Saves the cleaned sequences to 'rdp_regions_cleaned.fasta'.

```
#!/usr/bin/env python
# CLEANRDP.py: Remove gaps from RDP sequences
# Usage: CLEANRDP.py <trimmed_rdp.fasta>

import sys
from Bio.SeqIO import SeqIO

# Input file
input_file = sys.argv[1]

# Output file
output_file = "rdp_regions_cleaned.fasta"

# Clean sequences
with open(input_file) as in_handle, open(output_file, "w") as out_handle:
    for record in SeqIO.parse(in_handle, "fasta"):
        cleaned_seq = record.seq.replace("-", "")
        out_handle.write(f">{record.id}\n{cleaned_seq}\n")
```

This Python script checks and fixes issues with the FASTA file. It:

1. Checks the FASTA file for any errors (e.g., missing newlines, incorrect formatting).
2. Cleans the FASTA file, removing any unwanted characters or formatting.
3. Compares the cleaned FASTA file with a reference FASTA file to ensure only valid sequences remain.
4. Downloads missing sequences from NCBI (Protein database) and appends them to the 'cleaned_sequences.fasta' file.
5. Outputs the final 'cleaned_sequences.fasta' file.

CHECKFASTA.py

```
#!/usr/bin/env python
# CHECKFASTA.py: Check and fix FASTA file issues
# Usage: CHECKFASTA.py <cleaned_sequences.fasta>

import sys
import re

# Input file
input_file = sys.argv[1]

# Output file
output_file = "checked_sequences.fasta"

# Check and fix issues
with open(input_file) as in_handle, open(output_file, "w") as out_handle:
    for line in in_handle:
        line = line.strip()
        if line.startswith(">"):
            # Header line
            out_handle.write(line + "\n")
        else:
            # Sequence line
            if not re.match("^[ACGTUacgtu-]*$", line):
                # Invalid characters
                print(f"Invalid characters in sequence: {line}")
            else:
                # Valid sequence
                out_handle.write(line + "\n")
```

This Python script extracts a specific region (the RDP coding region) from aligned sequences using Biopython's AlignIO. It:

1. Reads the aligned sequences from 'aligned_sequences.fasta' using Biopython's AlignIO.
2. Finds each sequence to the trimmed_rdp.fasta specified region.
3. Writes the trimmed sequences to 'aligned_sequences.fasta'.

TRIMFASTA.py

```
#!/usr/bin/env python
# TRIMFASTA.py: Extract specific region from aligned sequences
# Usage: TRIMFASTA.py <aligned_sequences.fasta>

import sys
from Bio.AlignIO import AlignIO

# Input file
input_file = sys.argv[1]

# Output file
output_file = "trimmed_rdp.fasta"

# Extract region
with open(input_file) as in_handle, open(output_file, "w") as out_handle:
    alignment = AlignIO.read(in_handle, "fasta")
    start = 100000000
    end = 1000000000
    for record in alignment:
        subseq = record[start:end]
        out_handle.write(f">{record.id}\n{subseq}\n")
```

This Python script extracts a specific region (the RDP coding region) from aligned sequences using Biopython's AlignIO. It:

1. Reads the aligned sequences from 'aligned_sequences.fasta' using Biopython's AlignIO.
2. Finds each sequence to the trimmed_rdp.fasta specified region.
3. Writes the trimmed sequences to 'aligned_sequences.fasta'.

TRIMFASTA.py

```
#!/usr/bin/env python
# TRIMFASTA.py: Extract specific region from aligned sequences
# Usage: TRIMFASTA.py <aligned_sequences.fasta>

import sys
from Bio.AlignIO import AlignIO

# Input file
input_file = sys.argv[1]

# Output file
output_file = "trimmed_rdp.fasta"

# Extract region
with open(input_file) as in_handle, open(output_file, "w") as out_handle:
    alignment = AlignIO.read(in_handle, "fasta")
    start = 100000000
    end = 1000000000
    for record in alignment:
        subseq = record[start:end]
        out_handle.write(f">{record.id}\n{subseq}\n")
```

```
#!/usr/bin/env python
# TRIMFASTA.py: Extract specific region from aligned sequences
# Usage: TRIMFASTA.py <aligned_sequences.fasta>

import sys
from Bio.AlignIO import AlignIO

# Input file
input_file = sys.argv[1]

# Output file
output_file = "trimmed_rdp.fasta"

# Extract region
with open(input_file) as in_handle, open(output_file, "w") as out_handle:
    alignment = AlignIO.read(in_handle, "fasta")
    start = 100000000
    end = 1000000000
    for record in alignment:
        subseq = record[start:end]
        out_handle.write(f">{record.id}\n{subseq}\n")
```

GETSEQS.sh

This Bash script automates the retrieval of protein sequences in FASTA format from NCBI using accession numbers. It:

1. Defines a list of accession numbers.
2. Uses a `for` loop to fetch each sequence via the NCBI E-utilities API (`efetch.fcgi`) with `curl`.
3. Appends results to a file named `sequences.fasta`.

Output: A FASTA file containing all the requested protein sequences.

```
Project > $ getseqs.sh
1  #!/bin/bash
2
3  # This is a script used to download the accession files for use in the subsequent pipeline.
4  # The accessions are saved to a file called sequences.fasta (in fasta format)
5
6  # Define the list of accession numbers
7  accessions=(
8      "MG833031.1" "CAA25029.1" "BAA03151.1" "AAP97137.1" "NP_041277.1" "CAA25416.1" "NP_056777.1"
9      "BAA00168.1" "ABS84820.1" "YP_001285409.1" "NP_853560.2" "YP_145791.1" "AAQ64627.1"
10     "BAD06930.1" "NP_733845.1" "NP_277061.1" "NP_620564.1" "AAF80998.1" "AAN63804.2"
11     "AAF00472.1" "BAA32553.1" "NP_049374.1" "BAA21898" "AAC58807.1" "NP_046155.1"
12     "ADP24157.1" "YP_009337666.1" "YP_009337152.1" "YP_009336629.1" "ASK12212.1"
13     "APG77945.1" "ASK12194.1" "ASK12217.1" "APG77984.1" "LC310707"
14     "KU645789" "ABB17263.2" "YP_004063985.1"
15 )
16
17 # Output file for sequences in FASTA format
18 output_file="sequences.fasta"
19
20 # Loop through each accession number and download the sequence
21 for acc in "${accessions[@]}; do
22     echo "Fetching $acc..."
23     curl -s "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=protein&id=$acc&rettype=fasta"
24     sleep 1 # avoid rate-limiting
25 done
26
27 echo "All sequences downloaded and saved to $output_file."
28
```

CHECKERRORS.py

This Python script checks and fixes issues with a FASTA file downloaded using accession numbers. It performs three key tasks:

1. Cleans the FASTA file: Removes error lines starting with "+" to ensure only valid sequences remain.
2. Checking missing accessions: Compares the cleaned FASTA file with a list of accession numbers to identify any missing sequences.
3. Downloading missing sequences: Fetches missing sequences from the NCBI Nucleotide (nucore) database using `curl` and appends them to the final output file, fixed_sequences.fasta

```
Project > checkerrors.py > main
6 import os
7
8 # Paths to the original FASTA file and the new final output file
9 original_fasta_file = "sequences.fasta"
10 final_output_file = "fixed_sequences.fasta"
11
12 # Full list of accession numbers
13 accessions = [
14     "MG833031.1", "CAA25029.1", "BAA03151.1", "AAP97137.1", "NP_041277.1", "CAA25416.1", "NP_056777.1",
15     "BAA00168.1", "ABS84820.1", "YP_001285409.1", "NP_853560.2", "YP_145791.1", "AAQ64627.1",
16     "BAD06930.1", "NP_733845.1", "NP_277061.1", "NP_620564.1", "AAF80998.1", "AAN63804.2",
17     "AAF00472.1", "BAA32553.1", "NP_049374.1", "BAA21898", "AAC58807.1", "NP_046155.1",
18     "ADP24157.1", "YP_009337666.1", "YP_009337152.1", "YP_009336629.1", "ASK12212.1",
19     "APG77945.1", "ASK12194.1", "ASK12217.1", "APG77984.1", "LC310707",
20     "KU645789", "ABB17263.2", "YP_004063985.1"
21 ]
22
23 # Function 1: cleans original file of error outputs, which start with "+"
24 def clean_fasta_file(file_path):
25     with open(file_path, "r") as f:
26         lines = f.readlines()
27         cleaned_lines = [line for line in lines if not line.startswith("+")]
28     with open(file_path, "w") as f:
29         f.writelines(cleaned_lines)
30
31 # Function 2: checks if all accessions from the variable "accessions" are in the file. The missing ones are returned by the function.
32 def check_accessions(file_path, accessions):
33     with open(file_path, "r") as f:
34         content = f.read()
35     missing_accessions = [acc for acc in accessions if f">{acc}" not in content]
36     return missing_accessions
37
38 # Function 3: downloads missing accessions to end of file using the nucore database.
39 # We are assuming that if getseqs.sh missed a download, it is due to the accession not being in the protein database and is instead in the nucore database.
40 def download_missing_sequences(missing_accessions, final_file):
41     for accession in missing_accessions:
42         print(f"Downloading protein translation for CDS of {accession}...")
43         response = os.system(
44             f'curl -s "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucore&id={accession}&rettype=fasta_cds_aa&retmode=text" >> "{final_file}"'
45         )
46         if response == 0:
47             print(f"Completed download for {accession}.")
48         else:
49             print(f"Error downloading {accession}")
```

CLUSTAL.sh

This Bash script uses ClustalW to perform a multiple sequence alignment of protein or nucleotide sequences. It takes `fixed\_sequences.fasta` as the input file and generates an aligned output file named `aligned\_sequences.fasta` in FASTA format.

```
Project > $ clustal.sh
1  # ClustalW alignment of whole CDS for RdRp extraction.
2  clustalw -INFILE=fixed_sequences.fasta -OUTFILE=aligned_sequences.fasta -OUTPUT=fasta
3
```

Project > trimseqs.py > ...

```
1  from Bio import AlignIO
2
3  # input / output file
4  alignment_file = "aligned_sequences.fasta"
5  output_file = "trimmed_rdrp.fasta"
6
7  # Define the consensus-based start and end RdRp position (1-based indexing)
8  consensus_start = 3296
9  consensus_end = 3648
10
11 # Read the alignment
12 alignment = AlignIO.read(alignment_file, "fasta")
13
14 # Open output file for writing
15 with open(output_file, "w") as outfile:
16     for record in alignment:
17         # Extract the region corresponding to the consensus positions
18         trimmed_seq = record.seq[consensus_start - 1:consensus_end]
19         # Write the trimmed sequence to the output file
20         outfile.write(f">{record.id}\n{trimmed_seq}\n")
```

TRIMSEQS.py

This Python script extracts a specific region (the RdRp coding region) from a multiple sequence alignment based on consensus start (3296) and end (3648) positions. It:

1-Reads the aligned sequences from aligned_sequences.fasta using Biopython's AlignIO.

2-Trims each sequence to the specified region

3-Writes the trimmed sequences to trimmed_rdrp.fasta.

CLEANRDRP.py

This Python script removes gaps (-) from the trimmed RdRp sequences to ensure clean, gap-free sequences. It:

1. Reads the sequences from `trimmed\_rdrp.fasta` using Biopython's SeqIO.
2. Replaces gaps in each sequence with an empty string.
3. Saves the cleaned sequences to rdrp_regions_cleaned.fasta.

Project > cleanrdp.py > ...

```
1  from Bio import SeqIO
2
3  # input / output
4  input_file = "trimmed_rdrp.fasta"
5  output_file = "rdrp_regions_cleaned.fasta"
6
7  # open output file for writing, write trimmed file to new file replacing gaps (-) with ""
8  with open(output_file, "w") as output_handle:
9      for record in SeqIO.parse(input_file, "fasta"):
10         # Remove gaps from the sequence using string replacement
11         clean_sequence = str(record.seq).replace("-", "")
12         # Write the cleaned sequence to the new file
13         output_handle.write(f">{record.id}\n{clean_sequence}\n")
14
15
```

```
Project > $ tcoffee_rdrp.sh
1  #t-coffee alignment of RdRp regions.
2  t_coffee -infile rdrp_regions_cleaned.fasta -output fasta_aln -outfile rdrp_tcoffee_aligned.fasta
3
```

```
Project > $ clustal.sh
1  # ClustalW alignment of whole CDS for RdRp extraction.
2  clustalw -INFILE=fixed_sequences.fasta -OUTFILE=aligned_sequences.fasta -OUTPUT=fasta
3
```

CLUSTAL_RDRP.sh and **TCOFFEE_RDRP.sh**

At this stage of the pipeline, the process splits into two parallel paths to compare multiple sequence alignment methods: ClustalW and T-Coffee.

CLUSTAL_RDRP.sh:

Performs a multiple sequence alignment of the cleaned RdRp regions using ClustalW. It takes `rdrp_regions_cleaned.fasta` as input and outputs the aligned sequences to `rdrp_clustal_aligned.fasta`:

TCOFFEE_RDRP.sh

Uses T-Coffee to align the same cleaned RdRp regions. The input file is ``rdrp_regions_cleaned.fasta``, and the aligned output is saved as ``rdrp_tcoffee_aligned.fasta``.

MEGA PHYLOGENETIC TREES

Tree Construction in MEGA

Trees were made following the parameters set by the original paper:

Tree type:

Maximum Likelihood based on the JTT matrix-based model

Gaps and missing data:

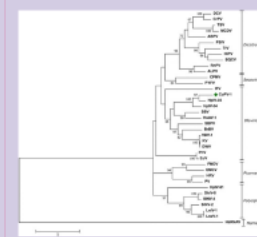
Complete deletion

Statistical significance:

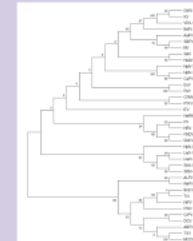
evaluated by 500 bootstraps

ClustalW tree comparison

Tree from the paper

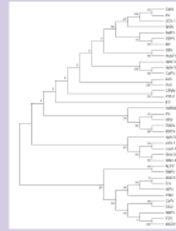


Our tree

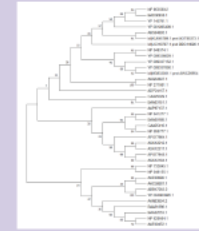


ClustalW vs. T-Coffee

Our ClustalW



Our T-Coffee



*Labels to be corrected on T-Coffee tree

Challenges with MEGA

The CLUSTALW trees don't match. This may be caused by

-Different MEGA versions (7.0.26 vs. 11.0.13)

-Tree type not listed (Original Tree vs. Bootstrap Consensus)

-Not stated whether MEGA was allowed to "Align" data or if original alignment was maintained.

Tree Construction in MEGA

Trees were made following the parameters set by the original paper:

Tree type:

Maximum Likelihood based on the JTT matrix-based model

Gaps and missing data:

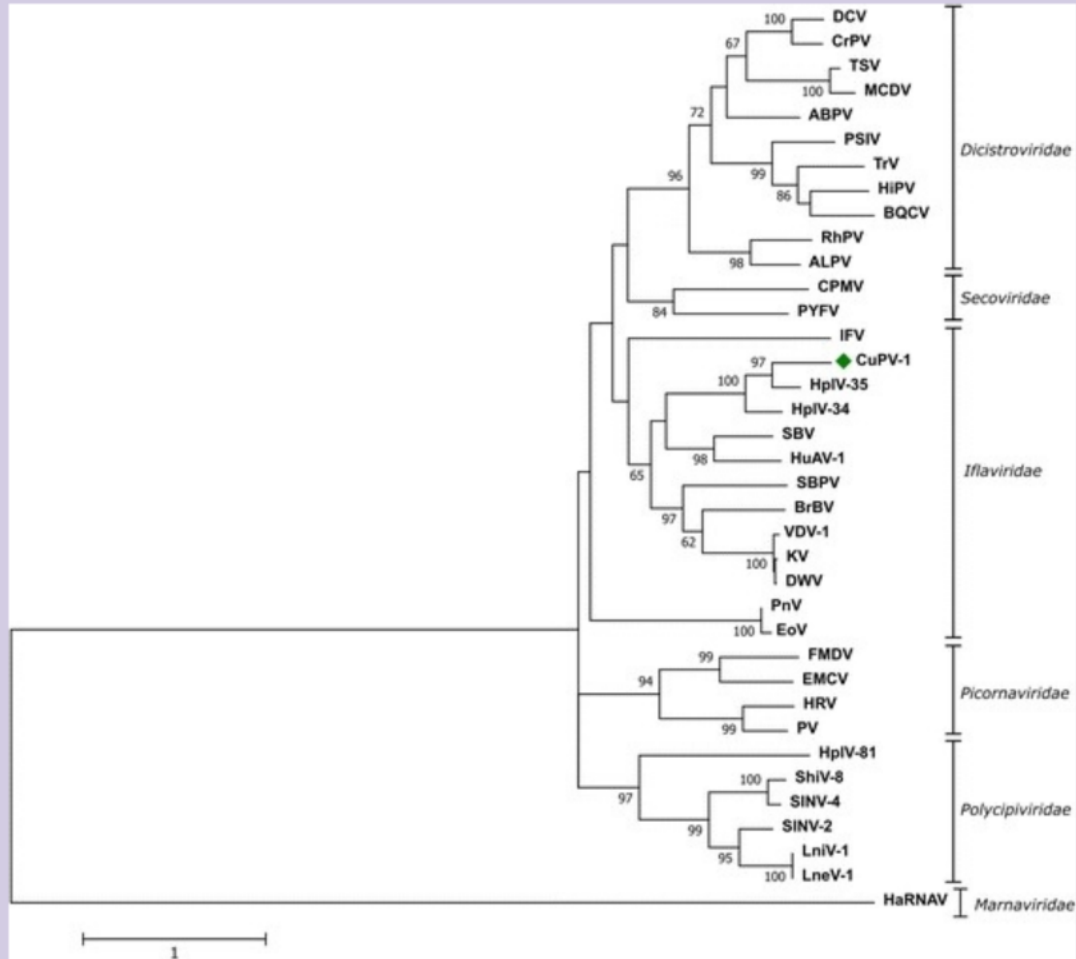
Complete deletion

Statistical significance:

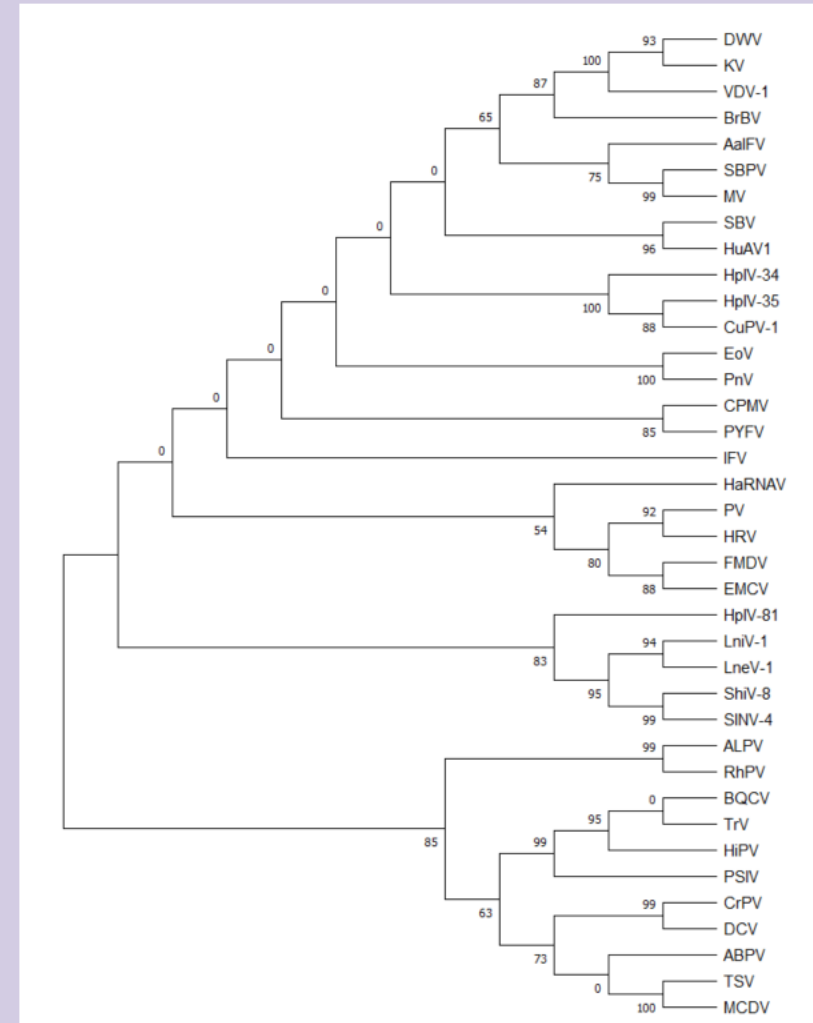
evaluated by 500 bootstraps

ClustalW tree comparison

Tree from the paper

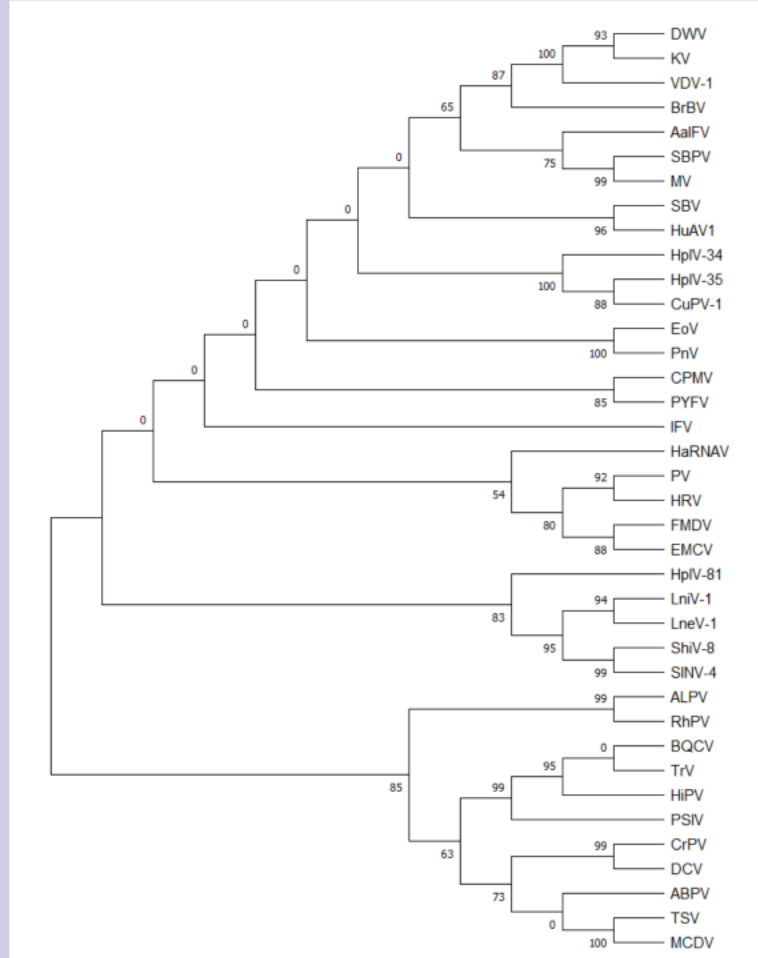


Our tree

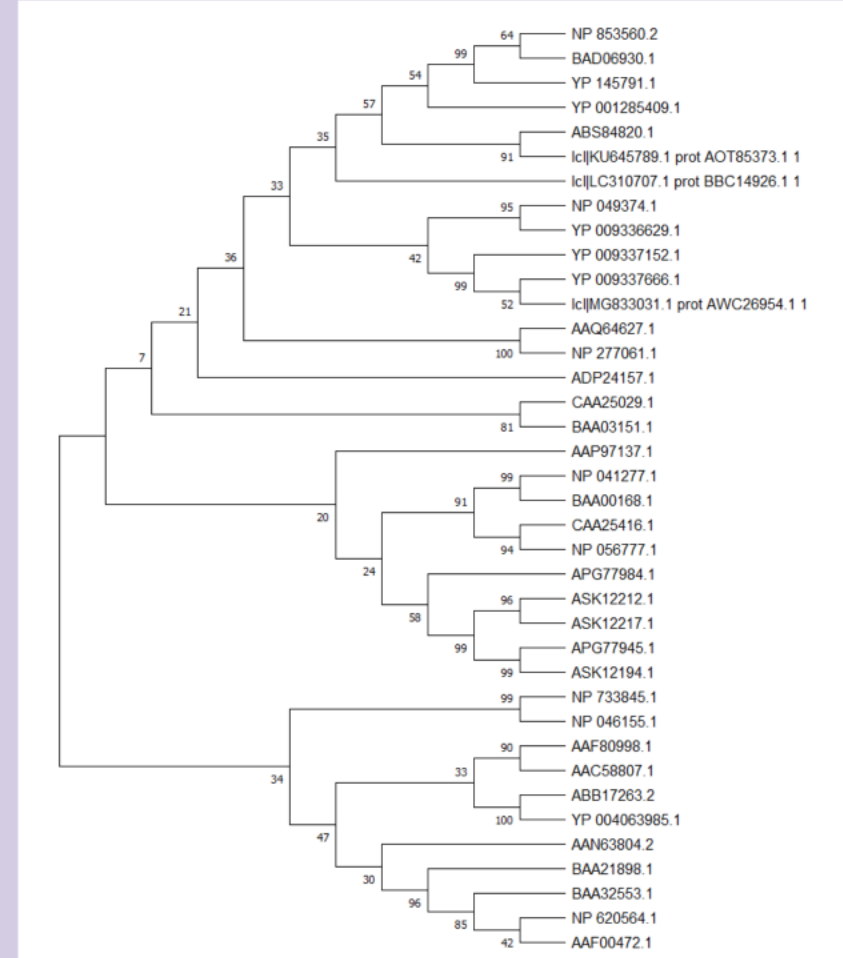


ClustalW vs. T-Coffee

Our ClustalW



Our T-Coffee



*labels to be corrected on T-Coffee tree

Challenges with MEGA

The CLUSTALW trees don't match. This may be caused by

- Different MEGA versions (7.0.26 vs. 11.0.13)
- Tree type not listed (Original Tree vs. Bootstrap Consensus)
- Not stated whether MEGA was allowed to "Align" data or if original alignment was maintained.

So, that didn't work...

A statement on reproducibility

This project is a testament to the importance of reproducibility. We faced many struggles due to the paper we followed not reporting certain things.

We ended up with a completely different result. This is not ideal!



CLUSTALW



T-COFFEE



Our attempts to fix it:

(we had to deviate from the "pipeline")

Used EMBL-ELI Pfam:

Aligns sequences to predefined hidden Markov models (HMMs) to detect conserved regions associated with known protein families.

Extracted **SUPERFAMILY SSF56672** (RDRP) using script

Realigned with both **CLUSTALW** and **T-Coffee**

-Phylogenetic tree made using IQTREE:

A software for constructing maximum-likelihood phylogenetic trees, offering model selection, ultrafast bootstrapping, and efficient tree search algorithms.



Final thoughts

We want to find a way to automate this process

It will no longer parallel the chosen paper, just use their data.

We want to integrate the pipeline into Nextflow

Reproducibility is important!

A statement on reproducibility

This project is a testament to the importance of reproducibility. We faced many struggles due to the paper we followed not reporting certain things.

We ended up with a completely different result. This is not ideal!



Our attempts to fix it:

(we had to deviate from the "pipeline")

Used **EMBL-ELI Pfam:**

Aligns sequences to predefined hidden Markov models (HMMs) to detect conserved regions associated with known protein families.

Extracted **SUPERFAMILY SSF56672** (RDRP) using script

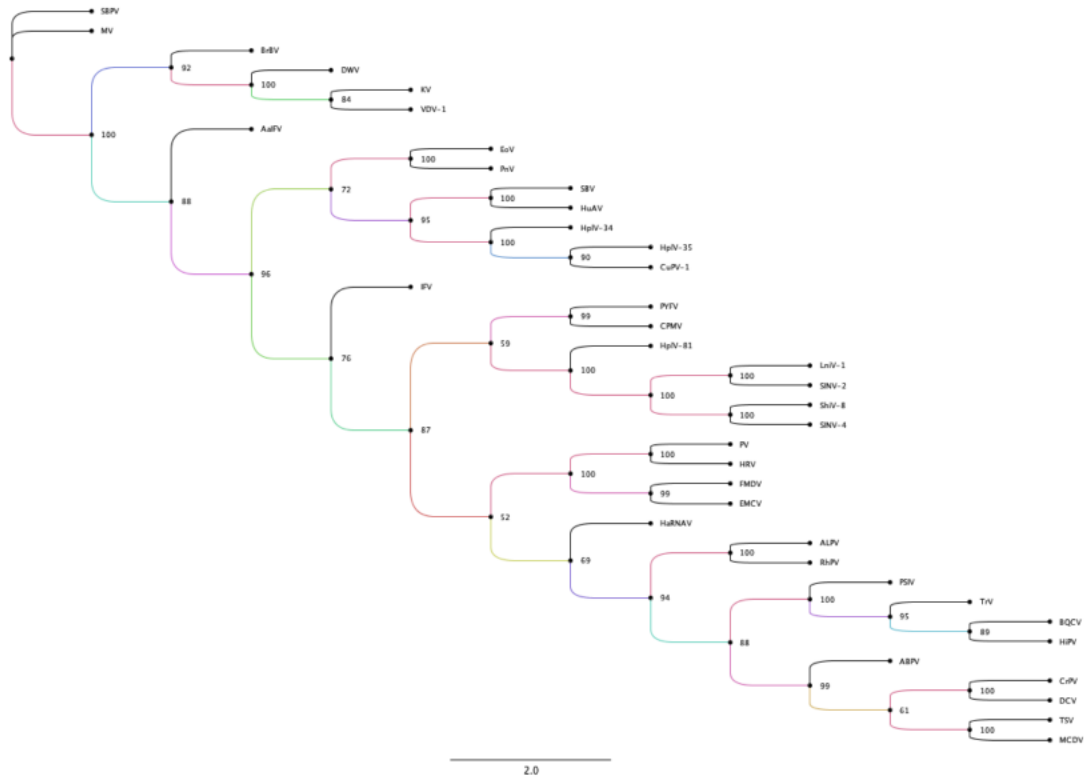
Realigned with both **CLUSTALW** and **T-Coffee**

-Phylogenetic tree made using **IQTREE:**

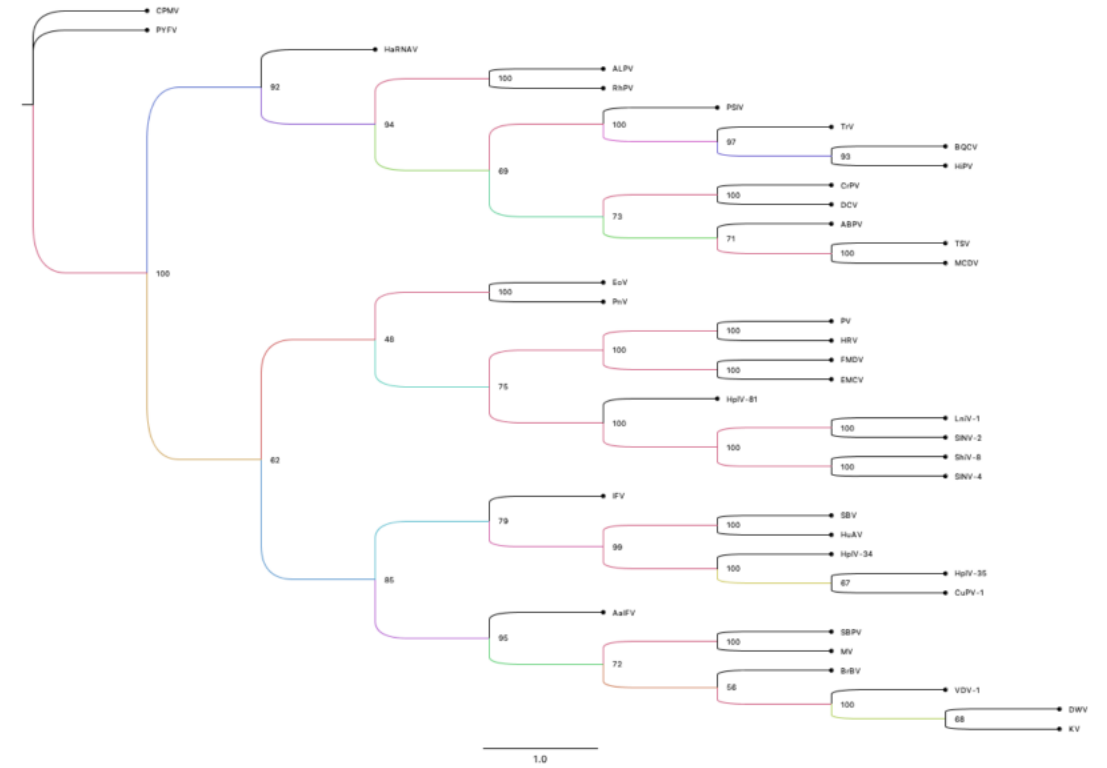
A software for constructing maximum-likelihood phylogenetic trees, offering model selection, ultrafast bootstrapping, and efficient tree search algorithms.



CLUSTALW



T-COFFEE



Final thoughts

We want to find a way to automate this process

It will no longer parallel the chosen paper, just use their data.

We want to integrate the pipeline into Nextflow

Reproducibility is important!